

7. Gün Notlarım: J1939 Mesajları Nasıl Kodlanıyor?

Bugün gerçek bir CAN mesajının içine bir aracın hızının nasıl gizlendiğini öğrendim.

Yani "bir sayı olan hız bilgisi, nasıl oluyor da 8 byte'lık ham bir veriye dönüşüyor" sorusunun cevabını buldum.

CAN ID nedir, neden bu kadar uzun?

Her CAN mesajının bir kimlik numarası var, buna CAN ID diyoruz. J1939'da bu ID 29 bit uzunluğunda ve içinde birden fazla bilgi aynı anda taşınıyor. Yani tek bir sayı değil, parçalara bölünmüş bir sayı:

- İlk birkaç bit öncelik (priority) bilgisini taşıyor — hangi mesajın hatta önce gireceğini belirliyor, sayı ne kadar küçükse öncelik o kadar yüksek.
- Ortadaki kısım PGN dediğimiz, mesajın "hangi konu hakkında olduğunu" söyleyen kısım. Mesela PGN 65265 dediğimizde bu "araç hız/durum bilgisi" mesajı demek.
- En sonda da mesajı kimin gönderdiği yazıyor (kaynak adresi / source address).

Yani hangi araç ya da hangi ECU bu mesajı yolladı.

Ben burada şunu anladım: CAN ID rastgele bir sayı değil, kendi içinde anlamlı bölümlere ayrılmış bir "adres etiketi" gibi çalışıyor.

PGN 65265 ne işe yarıyor?

PGN 65265, aracın hızı, freni, el freni, cruise control gibi bilgileri taşıyan standart bir mesaj türü. Buna kısaca CCVS1 deniyor. Her araç bu mesajı saniyede 10 kere (100 ms'de bir) yayınlıyor, böylece hattı dinleyen herkes o an aracın hızının ne olduğunu anlık olarak görebiliyor.

Hız bilgisi nasıl sayıya çevriliyor?

Burası benim için en ilginç kısımdı. Hız km/h cinsinden bir ondalıklı sayı (örnek: 87.5 km/h), ama CAN mesajı sadece byte'lardan (0-255 arası tam sayılardan) oluşuyor. Ondalıklı bir sayıyı byte'lara nasıl sığdırıyoruz? Cevap şu: hızı 256 ile çarpıyoruz ve yuvarlıyoruz. Yani:

```
ham_deger = hiz_kmh * 256
```

Çıkan sayı 2 byte'a bölünüp (biri düşük, biri yüksek byte) mesaja yerleştiriliyor.

Karşı taraf bu iki byte'ı birleştirip 256'ya bölünce orijinal hızı geri elde ediyor.

Bu yönteme "ölçek/çözünürlük" (resolution) deniyormuş — burada çözünürlük 1/256 km/h, yani en küçük hissedilebilir hız farkı yaklaşık 0.0039 km/h.

Örnek olarak 60 km/h'yi ele alırsam:

```
60 * 256 = 15360 -> hex: 0x3C00
düşük byte: 0x00, yüksek byte: 0x3C
```

Bu iki byte mesajın 2. ve 3. byte'ına yazılıyor, gerisi sabit kalıyor.

Ayrıca öğrendiğim bir şey de şu: bazı özel değerler "veri yok" ya da "hata" anlamına geliyor (örneğin 0xFFFF). Bu yüzden gerçekte kullanılabilecek maksimum hız biraz daha düşük tutuluyor (yaklaşık 251 km/h), üstü hata/boş veri bandına ayrılmış.

30 araçlık bir filoyu nasıl temsil ettim?

Her aracın kendine ait bir kaynak adresi (source address) olması gerekiyor, yoksa hangi mesajın hangi araçtan geldiğini ayırt edemeyiz. Ben de 10 farklı marka, her markadan 3 model olacak şekilde 30 araç tanımladım ve her birine benzersiz bir adres verdim. Bunu ayrı bir JSON dosyasında tuttum ki kod ile veri birbirine karışmasın.

Kodu nasıl organize ettim?

Mesajı byte'lara çeviren kısmı (encode/decode mantığı) tamamen ayrı bir dosyada tuttum ve bu dosyanın hiçbir şekilde ağ, dosya veya donanımla konuşmamasına dikkat ettim. Sebebi şu: bu şekilde bu fonksiyonları test etmek çok kolaylaşıyor, çünkü sadece sayı veriyorsun, sayı alıyorsun — arada başka hiçbir şey yok.

GitHub Actions ile ilk tanışmam

Bugün ayrıca GitHub Actions'ı da öğrenmeye başladım. Basitçe şunu anladım: depoya kod gönderdiğimde (push ya da pull request attığımda) GitHub'ın kendisi otomatik olarak benim yerime bazı komutları çalıştırabiliyor — mesela testleri çalıştırmak gibi. Buna workflow deniyor.

Bir workflow'un içinde şu yapı var:

- Workflow: `.github/workflows/` klasöründeki bir YAML dosyası, en dıştaki kutu.
- Job: workflow'un içindeki, kendi başına bir bilgisayarda (runner) çalışan iş

parçası. Bir workflow'da birden fazla job olabilir, bunlar genelde aynı anda (paralel) çalışır.

- Step: bir job'un içindeki sıralı adımlar. Her adım ya hazır bir "action"

kullanır (örneğin kodu indiren `actions/checkout`) ya da doğrudan bir komut çalıştırır (`run: pytest`).

Ve bunu tetikleyen şey `on:` kısmı — "ne zaman çalışsın" sorusunun cevabı. Push olunca mı, pull request açılınca mı, yoksa belirli bir saatte mi?

Basit bir örnek üstünde denedim:

```
name: CI
on: [push, pull_request]
jobs:
```

```
test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: echo "Merhaba GitHub Actions"
```

Bunu anlayınca, kendi projemdeki `.github/workflows/main.yml` dosyasının aslında korkunç bir şey olmadığını, sadece bu üç kavramın (workflow, job, step) üst üste tekrarı olduğunu fark ettim. Detaylara (test/build/deploy aşamaları) 9. günde girdim.